1. parseCompoundStmt

handles a block of curly braces. matches the opening brace, then loops parsing declarations first and then statements until it hits the closing brace. just calls parseDecl() repeatedly till it returns null, then parseStmt() repeatedly the same way.

2. parseConstantExpr, parseConstantTerm, and parseConstantOperand

ConstantExpr handles +/-, ConstantTerm handles */÷/%, and ConstantOperand is the base case which is either a literal number or a ConstantExpr within parentheses. this allows order of operations to be implemented as well

3. parseIncFactor and parseDecFactor

handle prefix increment (++x) and decrement (--x). check for the ++/-- token, then require an identifier right after it (returns null otherwise). return the retval as ASTIncExpr or ASTDecExpr as required

4. parseValue

handles the optional '!' before a factor. if current token is '!', consume it and require a factor afterwards, wrap in ASTNotExpr. otherwise just passes through straight to parseFactor().

5. parseTerm and parseTermPrime

handles multiplication-level precedence (*, /, %). parseTerm gets the first operand via parseValue, then parseTermPrime recursively builds up the ASTBinaryMathOp chain for each *, /, % it finds, checking the rhs isnt null each time or it throws OperandMissing.

6. parseNumExpr/Prime

same idea as Term but one level up — handles +/-. parseNumExpr calls parseTerm for its first operand (so */÷ binds tighter than +/-, normal precedence stuff), then Prime recursively chains any +/- ops it finds.

7. parseRelExpr/Prime

handles comparison operations. parseRelExpr starts by calling parseNumExpr so math binds tighter than comparisons, then Prime builds the ASTBinaryCmpOp chain, throwing OperandMissing if a comparison is missing its right side.

8. parseAndTerm/Prime

handles &&, one level below || in precedence. calls parseRelExpr first so comparisons bind tighter than && does, then Prime recursively builds ASTLogicalAnd nodes, same OperandMissing check pattern as the others.

9. parseWhileStmt

checks for while keyword, matches (, requires a valid condition expr (throws otherwise), matches ‘)’, then parses the loop body with parseStmt(). wraps it all into an ASTWhileStmt.

10. parseIfStmt

checks for if, matches ‘(‘, requires a condition, matches ‘)’, parses the then-branch with parseStmt(), and optionally checks for an else keyword to parse an else-branch too. builds an ASTIfStmt with both branches (else can be null).

11. parseAddrOfArrayFactor

handles array indexing — grabs the address of an array element. checks for &, requires an identifier after it, then requires at least one [expr] subscript (throws if missing), and can chain more for multi-dim arrays like 2d or 3d. wraps the final ASTArraySub in an ASTAddrOfArray node.

12. parseForStmt

parses for loop init, cond, step body. init and step both go through parseAssignStmt (with a flag telling it whether to eat the trailing ‘;’), cond goes through parseExpr, and body goes through parseStmt. had to add a fallback to parseExpr() for step specifically since things like prefix increment aren't handled by parseAssignStmt. any missing statement gets passed as null to the ASTForStmt constructor.

13. parseDoWhileStmt

parses do body while(cond);. body comes before the condition here unlike a normal while loop as the loop is supposed to run once before condition is checked. parses the body first and saves it, then matches while, ‘(‘, the condition expr, ‘)’, and then trailing semicolon.

14. parseSwitchStmt & parseCaseStmt

parseCaseStmt handles one case{} or default: block, using parseConstantExpr for the label since it has to be a compile time constant, then loops calling parseStmt() to grab the block's statements until it hits the next case, default or ‘}’. also peeks for break before each statement to set the case's break flag. parseSwitchStmt wraps the whole switch and just calls parseCaseStmt() repeatedly to collect all the case blocks.

15. parseInitializerList

parses the ‘{}’ after an ‘=’ in an array declaration matches the opening brace, then loops parsing comma-separated elements. each element is either a normal expr, or (for multi-dim arrays) another nested { } via calling itself recursively. adds each one with addElem(), stopping at the closing brace.

16. synchronize

error recovery function used after a catch block. eats tokens one at a time until it hits either something in the given syncSet, a token that starts a new statement, or EOF (which throws since theres nothing to recover into). if it lands on a semicolon and consumeTerminator is true it eats that too, but leaves closing braces/keywords alone so the caller can still match them properly.